Agent Powered Robot for Adaptive Navigation in Dynamic Environments

Anirudh Kaluri (akaluri) ,Brennen Farrell (btfarre2), Miles Daly (mtdaly)

Abstract: Traditional ROS2 powered robots use the Nav2 navigation stack to move the robot around a world. However, the advent of Large Language Models along with visual LLMs (vLLMs), the world in which the robot navigates can be perceived and understood more vividly. In this paper, we try to leverage "LLM" to navigate the robot using an Agentic framework. The LLM will "drive" the robot and try to detect obstacles too.

1. Introduction

Over the past few years, LLMs have become widespread in almost every field. Companies such as OpenAI, Anthropic, and others have created models that are both fast and relatively cheap to use.

Robot locomotion and route planning has seen significant overhauls with the rise of machine learning and large language models. Companies such as Tesla and Waymo have shown this through their self-driving cars. An approach using LLMs allows for more dynamic decision making rather than rigid heuristics. This can bring great benefits in performance over environments that vary from the pre-programmed expectations.

Previous work using LLMs to path-plan robots has been done using on board sensors, primarily LiDAR. Streaming all this data to an LLM can be expensive, as it requires the LLM to interpret the data and reconstruct an "image". Sending a top down image of the robot to an LLM can save the initial processing state as it is already in a format an AI is well trained on. The following are the contributions of the team-members in building the project:

Team Member	Work
Anirudh kaluri	1) Rasterization of Gazebo worlds 2) Agentic Robot steer flows
Brennen Farrell	1) Project structure setup 2) Aligning Gazebo and RVIZ frames of references 3) Nav2 baseline flow 4) Topography based Robot steering flow
Miles Daly	1) Experimentation package setup 2) Logging setup

2. Related Works

Researchers have experimented with using LLMs to influence robot behaviors in pathfinding and motion. In Baumann et al. [1], researchers from ETH Zurich used these technologies to have an autonomous driving system translate human commands into real-world actions. This system will take human instructions, compare them to the current machine state, and decide if the two are aligned. If not, then it sends directions to an MPC (Model Predictive Control) controller, which acts as a local controller for vehicle movement by optimizing control actions by picking the best driving actions to minimize directional error while not breaking any safety rules. In response to the LLM, the elements penalized or promoted by the MPC can change. This, in turn, can modify steering, acceleration, or braking in order to adjust behavior to reach the desired state. For example, if the vehicle is told to “Drive smoothly”, then the robot may attempt to avoid rapid acceleration and sharp steering in favor of more gradual motion.

Further methods have attempted to address the routing, rather than the behavior. In Xie et al. [2], researchers used a topological map to help guide the LLM routing. By translating the map into a form easily processable by the LLM, the robot is able to perform more flexible and human-like path planning rather than relying solely on coordinates. This is one of the easiest ways to bridge the gap between navigation and reasoning. This use of LLMs greatly assists in making pathfinding less bespoke. In Tariq et al [3], a robot uses human commands, LiDAR sensors, and JSON waypoints to navigate across a map. Unlike a traditional heuristic or deep reinforcement learning methodology, this system does not require extensive retraining for new environmental configurations. Additionally, the use of sensors such as LiDAR assists in detecting obstacles and replanning a route in real time.

Work has also been done using LLMs to choose existing path following algorithms. In Kim et al. [4], they used an LLM to decide which algorithm to use based on the density of obstacles, path length, and available resources. The LLM processed top down images to determine where the obstacles are and which kind of path would be best.

In the past, the process of routing and pathing a robot has been done in multiple stages using different technologies put together. Shah et al. [5] published the first use of pre-trained LLMs to determine actions without fine tuning. They took human readable text and converted it into intermediate locations using an LLM. They then used a Vision Language Model to get the probability of an image being the text extracted from the LLM, using a camera on board their robot. Finally they used a Visual Navigation Model to path the robot based off captured images.

These works demonstrate how LLMs enable robots to operate at a higher level of reasoning, allowing for more flexible and adaptive behavior. In comparison to heuristic models or neural networks, which often require extensive retraining, especially when introduced to a new environment, LLM-based approaches generalize more effectively to new situations. By translating human instructions into structured representations such as cost functions, maps, or waypoints, LLMs allow robots to interpret goals and adjust their actions based on context.

3. Methodology

The idea is to capture a top view of the map. This map will have information about the position of the robot and its destination. The LLM will use this map to guide the robot using navigation tools provided to it. For example, Figure 1 depicts an image which shows a robot traversing along the white corridors. The LLM understands the image and invokes tools to move forward and rotate.

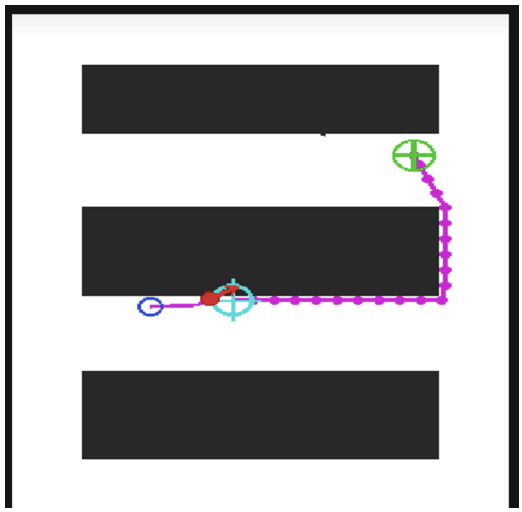

Figure -1

The entire project was setup through different stages:

a) Rasterizing the Gazebo Worlds
b) Generating test end points
c) Developing different techniques for LLMs to navigate the robot
d) Establish experimentation package to run the experiments automatically and log the results

3.1. Rasterizing the Gazebo World's

In this project, LLM needs a top-view image of the world/map in which the robot is travelling. Based on this image, the LLM will guide/steer the robot to the destination. Gazebo is a physics simulation tool which renders a 3D world- an environment in which the robot can traverse and interact. However, the tool is not friendly to extract data during runtime.

RVIZ renders 2D maps called "Portable Grey Maps" depicting the entire world from a top-view in a 2D image. The PGM is a scaled image of the Gazebo world. In other words, a robot travelling a given number of metres in gazebo world will travel the same number of pixels in the PGM image. Hence, RVIZ renders a PGM/PNG image that can be sent to an LLM to understand the robot's current position and the destination accurately. The LLM can then decide to go forward or rotate.

In order to achieve this, we have chosen a warehouse gazebo world (https://github.com/leonhartyao/gazebo_models_worlds_collection/blob/master/screenshots/warehouse.jpg) and rasterized it to an RVIZ friendly PGM format. The *world_to_map* package in the project rasterizes the XML type Gazebo world. It allocates a white image first and then projects the shape by converting the world coordinates into pixel coordinates based on a translation. It also provides a side-car that helps the

world to map offset so that the RVIZ grid can stay aligned with the Gazebo coordinate frame. For example RVIZ helps Rasterize Warehouse as shown in Figure 2.

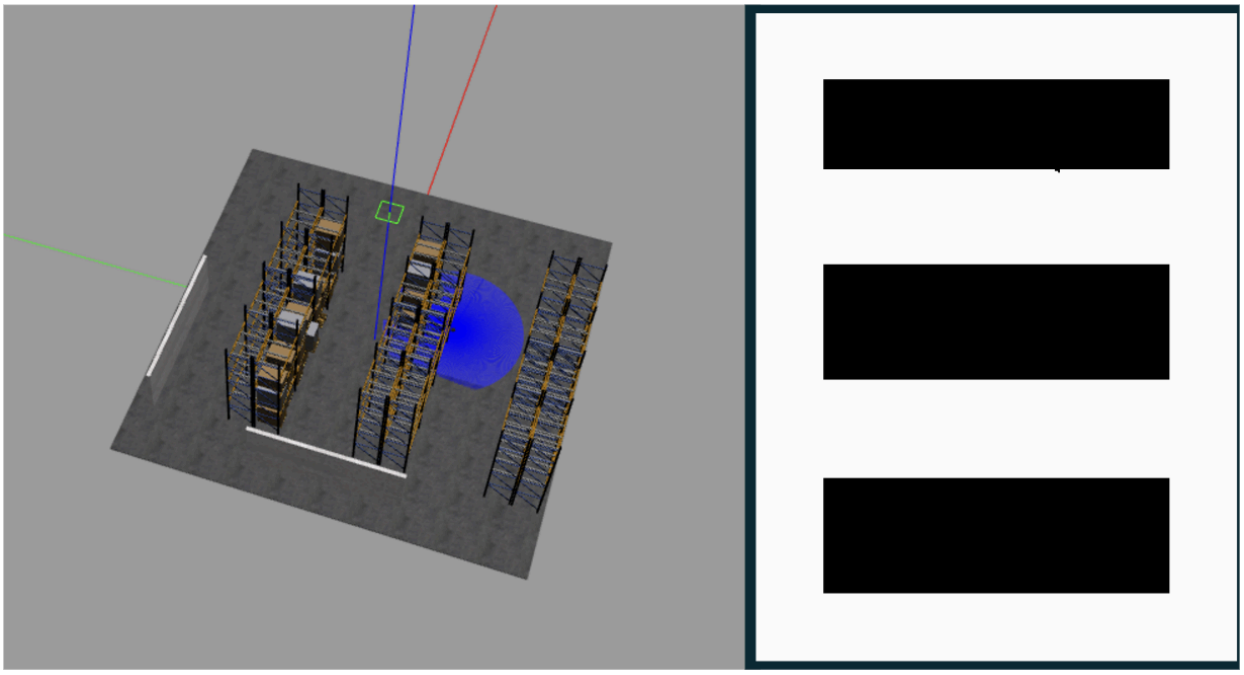

Figure-2

3.2. Generating the test endpoints

Another big advantage of using RVIZ compatible PGM maps is to generate coordinates in the map or world which can be used for experimentation and test cases. By using the “2D Publish Point” tool as shown in Figure-3 in RVIZ, we can capture the coordinates using the /clicked_point topic in ROS2. These points are then used as source and destination points for our robot.

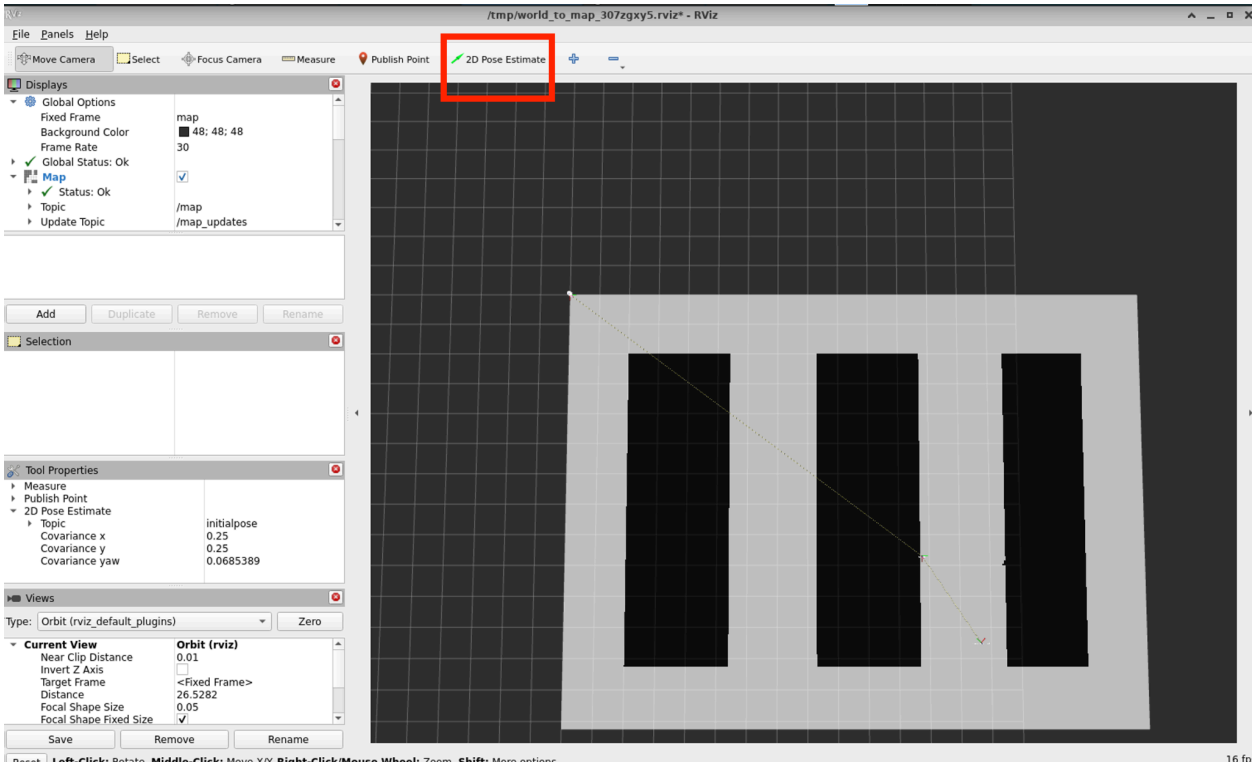

Figure-3

3.3. Different AI Flows to control the Robot

In this project, LangChain and LangGraph tools were used to build Agentic navigation system to control the robot. A total of 5 solutions were developed where four of them were powered by Agents whereas the fifth flow is a baseline Nav2 powered destination pursuit system. The source code for all the flows can be found at : https://github.com/csc591-robotics/intro/tree/main/src/nav2_llm_demo/nav2_llm_demo/llm

The agents use the following tools:

a) get_map_view() to get the 2D top view of the map
b) move_forward() to move the robot forward by x metres
c) rotate(degrees) to rotate the robot by x degrees
d) get_robot_pose() to get the Position and orientation of the robot
e) get_situation() : provides both map and lidar perception
f) check_goal_reached() to check if the current robot position is within the goal threshold precision.

Each of the flows are discussed below:

3.3.1 LangGraph ReAct with Multimodal Tool Results

The robot is powered by LangGraph's prebuilt create_react_agent(). The ReAct loop - model call, tool execution, model call - runs until the model reaches the destination or exhausts the 50 recursion limit. History compaction and image pruning are applied to minimize the context size as current APIs are limited by the size of the payload. This flow only uses the 2D top view of the

map to understand the robots position. The LLM is requested to make sure the robot doesn't navigate/traverse into obstacles in the map that are represented by the black pixels.

3.3.2 ReAct with Map and LiDAR fusion

This flow extends the previous one by adding live LiDAR data as an additional information channel. The lidar data is captured using the */scan* topic in ROS2. It is processed by a *lidar interpreter* which will provide a natural language explanation of the surroundings by processing the lidar data. This data will then reach another agent which makes a decision- whether to rotate or move forward.

3.3.3 A* Global Planner with LLM Path Follower

This flow shifts the route planning responsibility from the LLM to a relatively more deterministic A* algorithm. However, the steering controls of the robot are still provided as tools to the Agent so that they can navigate the path. Initially, the PGM occupancy map is loaded and provided as an input to A* with euclidean heuristic from the source pixel to the destination pixel. A magenta polyline is drawn on this map to indicate the path that should be taken by the robot. The current position of the robot and the destination are both marked on this map for every LLM call. Any drift in the robot's heading is calculated and given as a context to the LLM which then steers the robot on to the right path.

3.3.4 Topology Graph + LLM route planner

In this flow we split where to go from how to move. A topology graph is built from the occupancy map and the LLM only returns a JSON path of node IDs. This sequence of nodes help robot navigate from the current node to the goal. A separate ROS node drives the graph edge with a deterministic controller (either rotate or move forward). If the traversal fails, the edge is marked blocked and the LLM is asked to plan again. The model again acts as a high-level route chooser over graph rather than low-level tool-loop agent.

3.3.5 Non-Agentic Baseline powered by Nav2

This flow uses Nav2's NavigateToPose action to send one goal to the destination. The Nav2 stack drives the robot. There is no LLM call. It is used as a baseline to compare other approaches

4. Results

The Agentic flows with LangGraph ReAct Agents (3.3.1 and 3.3.2) failed to converge. The following were the issues with the two flows:

a) Stuck in a loop executing the same actions Example: The robot kept revolving around itself when it hits an obstacle
b) The Agent lost the entire context of previously traverse path Example: The LLM started traversing in reverse going back along the same path
c) The Agent steered the LLM into obstacles despite multiple prompt engineering commands asking the agent to not drive the robot into a region populated with black pixels.

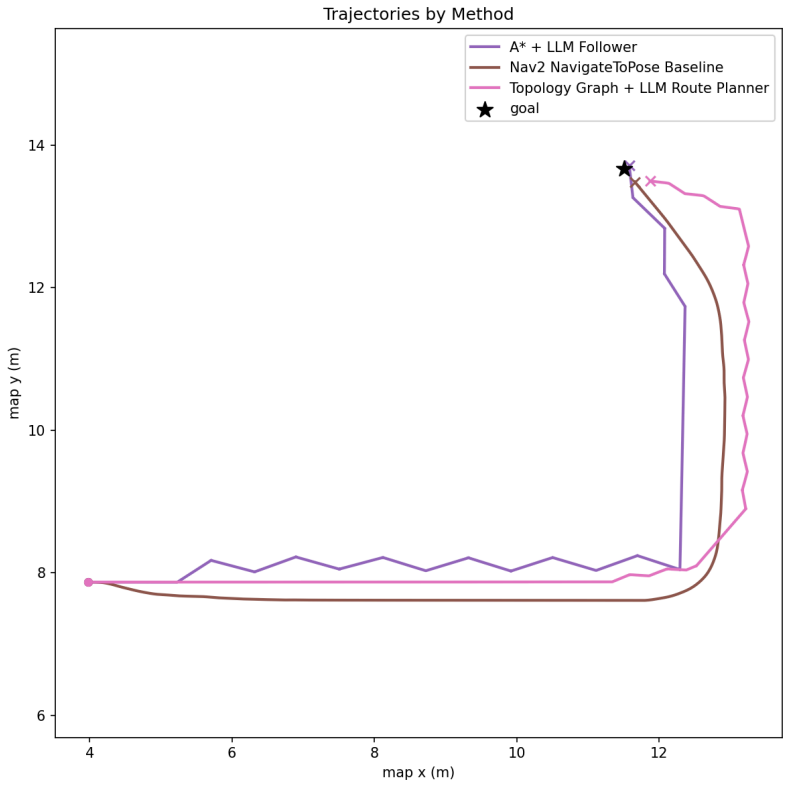

Figure 4: Trajectories by Method

Hence, for the purpose of evaluation, we aren't including the two flows. However, the experimental data for all the flows across 2 different experiments can be found here: https://github.com/csc591-robotics/intro/tree/main/experiment_data_folder. A trajectory of the path taken by the robot steered by three different flows is shown in Figure-6. The results are summarized in Table 1.

Method	Outcome	Runtime(s)	Distance Travelled (m)	Avg Speed (m/s)	LLM Calls/ Cycle
A* + LLM Follower	goal_reached	205.766	14.806	0.072	7
Nav2 NavigateTo Pose ***Baseline***	goal_reached	80.102	14.743	0.184	0
Topology Graph + LLM Route Planner	goal_reached	368.660	15.360	0.042	2

Table 1

The Nav2 NavigateToPose traversal method, which did not use AI, outperformed every other method across all measured metrics. It reached the goal in 80.1 s, covered a total distance of 14.743 m as shown in Figure-5 and Figure-6, and achieved an average speed of 0.184 m/s. This made it the fastest and most efficient approach in our experiment. Among the LLM-based methods, the A* + LLM Follower demonstrated the strongest performance. It completed the run in 205.8 s, finishing just 0.082 m from the goal after traveling 14.806 m at an average speed of 0.072 m/s. In comparison, the Topology Graph + LLM Route Planner showed the lowest performance of the three approaches, requiring 368.7 s to complete the task, ending 0.398 m from the goal, and averaging 0.042 m/s over a total distance of 15.360 m. While all methods traveled a similar distance, their completion times and average speeds differed substantially.

In terms of LLM usage, the A* + LLM Follower made 41 calls during execution, whereas the Topology Graph planner made only 2. However, as it used these calls for route planning, it had to stop and wait for each call to resolve. This represents a significant difference in how frequently each method relied on LLM interaction. Despite the number of calls, the Topology Graph method recorded the longest completion time and lowest average speed among the evaluated approaches. This is likely a result of the topology graph relying on the LLM as a route planner, meaning it will not move until each call resolves, while the A* + LLM can move continuously despite LLM calls. These results highlight a tradeoff between the two approaches in both performance and LLM utilization. Additionally, in Figure 4 the Topology Map appears to travel further right than the other two. This is likely due to its method of travelling through points, rather than simply the most efficient method.

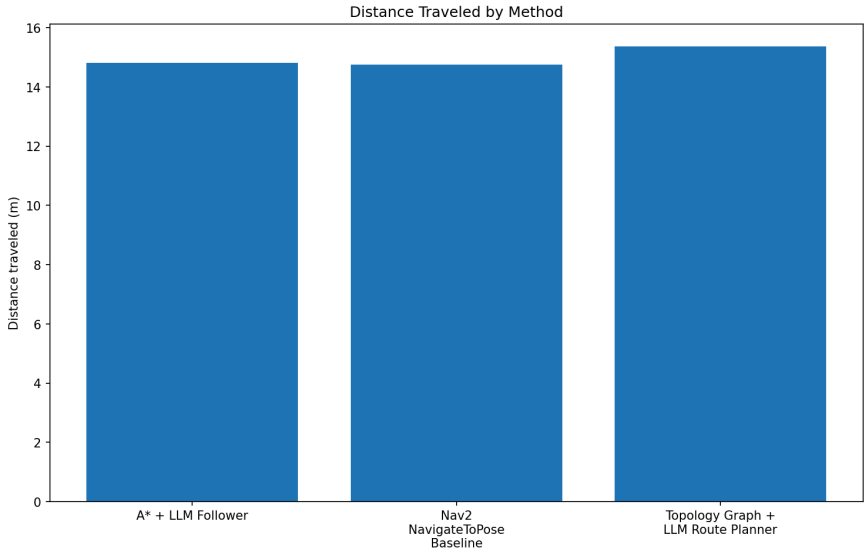

Figure 5: Distance Travelled By Method

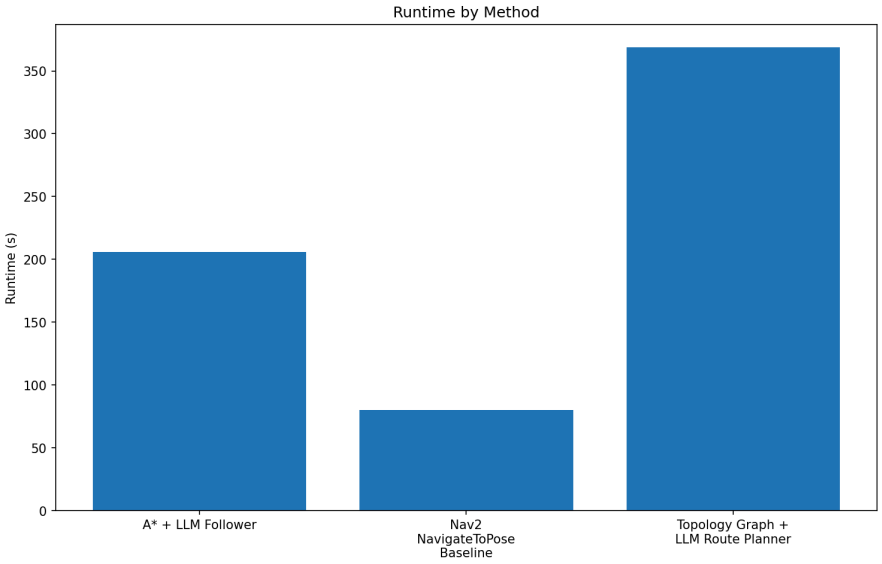

Figure 6: Runtime by Method

4.1 Lessons Learnt

The experiments and the project has provided a very broad scope for learning:

a) Choosing the right compute architecture is important at the beginning of the project. Gazebo and Rviz do not display on the Macbooks with ARM architecture. Even NVIDIA Isaac Sim works only on NVIDIA powered compute instances

b) Not all Gazebo worlds have Rviz counterparts. If we want to simulate them together, we have to rasterize a Gazebo world to get an Rviz map. The rasterization isn't straightforward and need many edge cases to project a 3D world into a 2D image
c) Aligning the frames of reference of Gazebo 3D world with corresponding RVIZ 2D world is a non-trivial problem. It requires additional details for origin and transformations so that the robot spawns at the same location in both tools when run simultaneously.
d) The current agentic APIs/frameworks are limited in their response structure. Specifically, an LLM model can only respond with text or tool calls. They cannot respond with an image. This was particularly evident when we attempted to draw the path on the image with the help of an LLM to guide the robot.
e) The LLMs cannot trace the path of the robot through previously sent sequences of top-view 2D images. The context window cannot accommodate large sequences of images indicating where the robot is on the map in top view.

5. Conclusion

Agentic AI frameworks allow LLMs to control the robots using the navigation interfaces as tools. These can be done with decent accuracy. However, LLMs cannot fully process a top view 2D image to trace a path from initial position to the goal or destination. The LLM loses context between the orientation of the robot and the map image. Hence, the robot frequently runs into obstacles. Shifting the planning responsibility to more deterministic approaches like A* while allowing the LLM to steer the robot significantly improved the accuracy and guided the robot to reach the goal. However, none of the approaches excelled Nav2 powered robot navigation to the goal with regards to runtime or distance travelled.

Citations

[1] N. Baumann, C. Hu, P. Sivasothilingam, H. Qin, L. Xie, M. Magno, and L. Benini, "Enhancing autonomous driving systems with on-board deployed large language models," arXiv preprint arXiv:2504.11514, 2025. doi: 10.48550/arXiv.2504.11514.

[2] F. Xie and S. Schwertfeger, "Empowering robotics with large language models: osmAG map comprehension with LLMs," arXiv preprint arXiv:2403.08228, 2024.

[3] M. Tariq, Y. Hussain, and C. Wang, "Robust Mobile Robot Path Planning via LLM-Based Dynamic Waypoint Generation," arXiv preprint arXiv:2501.15901, 2025.

[4] H. Kim, T. Lee, C. Jeong, and H.-S. Ahn, "Large Language Model as a Strategic Brain: Autonomous Selection of Path-Planning Algorithms for Mobile Robots," IEEE, pp. 727–732, Nov. 2025, doi: https://doi.org/10.23919/iccas66577.2025.11301388.

[5] D. Shah, B. Osinski, B. Ichter, and S. Levine, "LM-Nav: Robotic Navigation with Large Pre-Trained Models of Language, Vision, and Action," arXiv preprint arXiv:2207.04429, 2022.